

Prediction Market Data Pipeline

Kalshi and Polymarket ingestion, cross-platform analysis, and research synthesis for a weekly internship deliverable

Executive summary

I built a live data pipeline that pulls Kalshi and Polymarket prediction-market data into Supabase on a schedule, and paired it with research into how professional prediction-market traders think about pricing, resolution risk, and cross-platform edges. Roughly 19 markets across crypto, macro, and politics get polled on an unattended GitHub Actions cron job, nominally every 2 hours. On the analysis side, I used the collected data to build a repricing-speed metric, compare volume across markets, and measure price divergence across three linked market pairs. Along the way I caught a question-polarity mismatch that would have produced a false ~80-percentage-point "divergence" on the clearest pair in the dataset, and fixed it before it made it into a finding.

Everything below is a point-in-time analysis of data collected between **2026-09-02** and **2026-09-09** (640 snapshot rows across 19 markets). The cross-platform comparisons in Section 4.3 start from 2026-09-04, since Polymarket ingestion came online a day after Kalshi's. The pipeline keeps running, so the figures here describe that window and not the live state of the markets.

1. Architecture and pipeline

Schema

Six tables live in Supabase (Postgres), with Row Level Security enabled. Writes go through the `service_role` key, which bypasses RLS, so no read policies were needed for this stage of the project.

Table	Purpose
platforms	Kalshi / Polymarket
markets	One row per tracked market, upserted on (platform_id, external_id)
market_snapshots	The core time-series table: one row per market per poll
resolutions	Final outcomes (not yet populated, see Next Steps)
cross_platform_links	Manually matched market pairs tracking the same real-world event
kol_theses	Research log of prediction-market researcher theses and frameworks

Ingestion

Both platforms expose market and price data with no authentication required for reads. Only trading requires auth, and this pipeline never needed it. Kalshi's `/trade-api/v2/markets/{ticker}` and Polymarket's `/markets/keyset` both return the current price alongside metadata in a single call, so a periodic snapshot poll doesn't need a separate historical-candlestick or order-book call.

The watchlist (12 Kalshi tickers and 7 Polymarket slugs, across crypto, macro, and politics) is a fixed list rather than a dynamically re-selected top-N by volume. A market that drops in and out of tracking from day to day would break the time-series continuity the whole analysis depends on.

Automation and operations

A GitHub Actions workflow (`.github/workflows/prediction-markets-w1.yml`, at the repo root, since GitHub Actions doesn't discover workflows nested in a project subfolder) runs both ingestion scripts every 2 hours, or tries to, see below. It's offset from the top of the hour to avoid the queueing delays GitHub applies when many workflows are scheduled for :00 at once. A shared logger (`ingest/logging_config.py`) writes timestamped output to both the console, visible in the Actions logs, and a local `logs/pipeline.log` file, which the workflow also uploads as a build artifact so it survives past the runner. That replaced four separate, duplicated logging configurations that used to be scattered across the ingestion scripts.

The requested schedule is not the delivered schedule. Measuring the actual arrival times of 41 poll batches over 200 hours, the median gap between batches is 4.6 hours, not 2, with a worst observed gap of 34.6 hours. GitHub delivered roughly 41% of the scheduled runs (4.9 batches per day against 12 requested). Every batch that does land is complete, writing all 19 markets, so this is GitHub dropping or delaying scheduled events under load rather than the ingestion failing. The workflow reports green throughout, because the runs that execute genuinely succeed. Anything depending on a guaranteed interval would need a scheduler that isn't GitHub Actions cron, such as Supabase `pg_cron`, which was the alternative considered at the start.

After each ingestion run, a resolutions backfill step checks every tracked market against its platform and records the outcome once a market settles. The resolutions table is still empty since nothing tracked has closed yet, but the mechanism is in place and running. A second, independent workflow checks every 6 hours whether the most recent snapshot is over 4 hours old, and fails loudly, which triggers GitHub's normal failed-workflow notification, if the pipeline has gone quiet for longer than one missed run would explain.

2. Data sources: platform differences

- **Endpoint migration:** Polymarket's original `/markets` and `/events` endpoints are deprecated and were sunset on 2026-05-01. They still return HTTP 200, so it would have been easy to build the pipeline on a dying endpoint without noticing. I migrated to the replacement, cursor-paginated `/markets/keyset`, before writing any ingestion code.
- **Open interest isn't symmetric between platforms.** Kalshi exposes `open_interest` per market directly. Polymarket only exposes it at the event level, which aggregates across every sibling market in a multi-market event (all eight Bitcoin price-threshold markets share one event, for example). Attributing that combined number to a single market would misrepresent it, so Polymarket's `open_interest` is left NULL in this schema instead of populated with a misleading figure.
- **Price field choice:** Polymarket's `outcomePrices` field is the bid/ask midpoint, not the last executed trade price. I confirmed this directly against raw bid/ask data. The midpoint doesn't go stale on a quiet market the way "last trade" can, which makes it the better choice for a periodic snapshot.

- **Slugs aren't stable long-term identifiers** on Polymarket the way Kalshi tickers are. The same price threshold gets re-listed under a new slug once the prior instance resolves; I found three different "Bitcoin dip to \$60k" slugs for the same nominal question. Watchlist slugs need periodic re-verification because of this.
- **Matching strikes don't mean matching contracts.** While trying to link the tracked crypto markets across platforms, I found that Kalshi's ask what price BTC or ETH will be on one specific future date, while Polymarket's ask whether that price is ever touched before a deadline. Same asset, same rough dollar amount, different question, found by reading the contract terms rather than assumed from the strike price.

3. Research synthesis

I logged seven theses from two X accounts, spanning theory, trading methodology, and platform risk.

Framework, then evidence it's real

@**thenarrator** argues that a YES price near \$1 still carries a hidden cost: capital stays locked until settlement, so a near-certain bet can be capital-inefficient if resolution drags or the exit is thin. He thinks markets need a visible "carry" metric, something like expected time-to-redemption shown next to price, similar to a funding rate on perpetual futures. The last few cents near \$1, in his view, often just compensate for resolution or dispute risk the trader hasn't priced in.

@**Domahhhh** supplies four separate, documented cases where that abstract resolution risk actually played out. Polymarket resolved a Venezuela "invasion" market against its own stated criteria after Trump publicly claimed the US controlled the country. Polymarket resolved an Epstein "blackmail" market YES off one ambiguous email, using a markedly looser standard than it applied days later. Kalshi paid out a "will Biden meet Trudeau" market at \$0 even though the meeting was televised, because its rules only recognized two sources and neither reported it. And Kalshi settled an Oscars-viewership market on preliminary Nielsen numbers hours before the final numbers came out and flipped the outcome. Put together, these four cases turn narrator's abstract framing into something concrete and citable.

Trading methodology

@**Domahhhh's** Fed Chair nomination trade is a good example of this. He held a large net-short position against the market-implied favorite, who reached 85% consensus, for roughly five months and through four different frontrunner swaps, based on his own analysis that the favorite was a poor fit for monetary policy. The eventual pick matched his own, lower-probability estimate instead, netting him \$425,000. It's a real example of conviction against consensus pricing paying off, but only because the conviction was backed by genuine independent analysis rather than stubbornness.

Theory

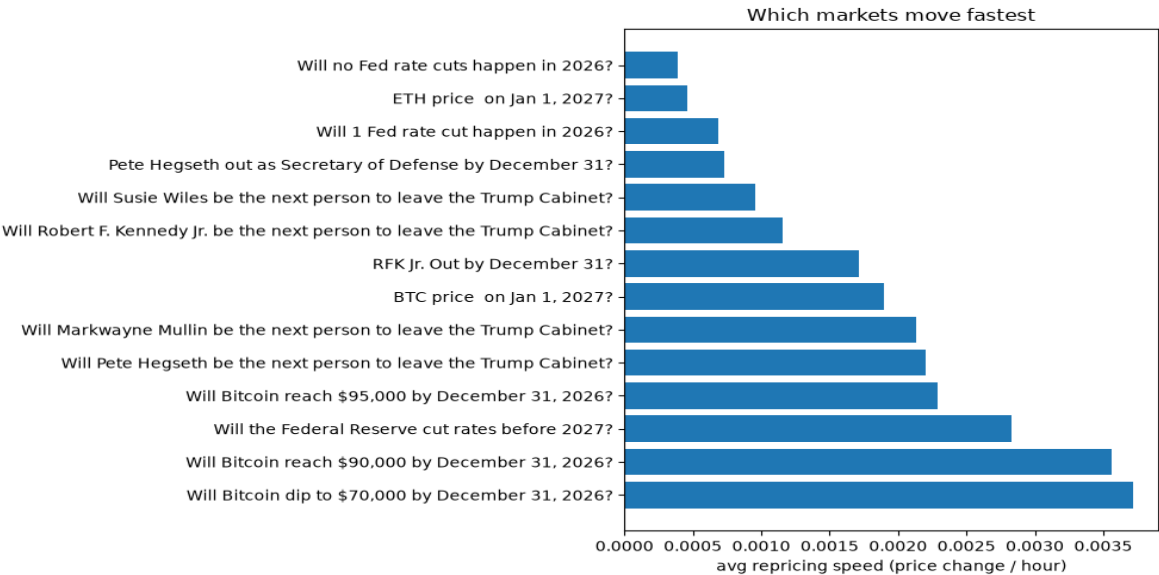
@**VitalikButerin** argues prediction markets are epistemically healthier than social media or unbounded asset markets. A bad take on social media earns unaccountable clout; a bad bet on a prediction market loses real money. And bounded $[0,1]$ pricing resists the reflexivity and pump-and-dump dynamics that

unbounded markets are prone to.

4. Findings from my own data

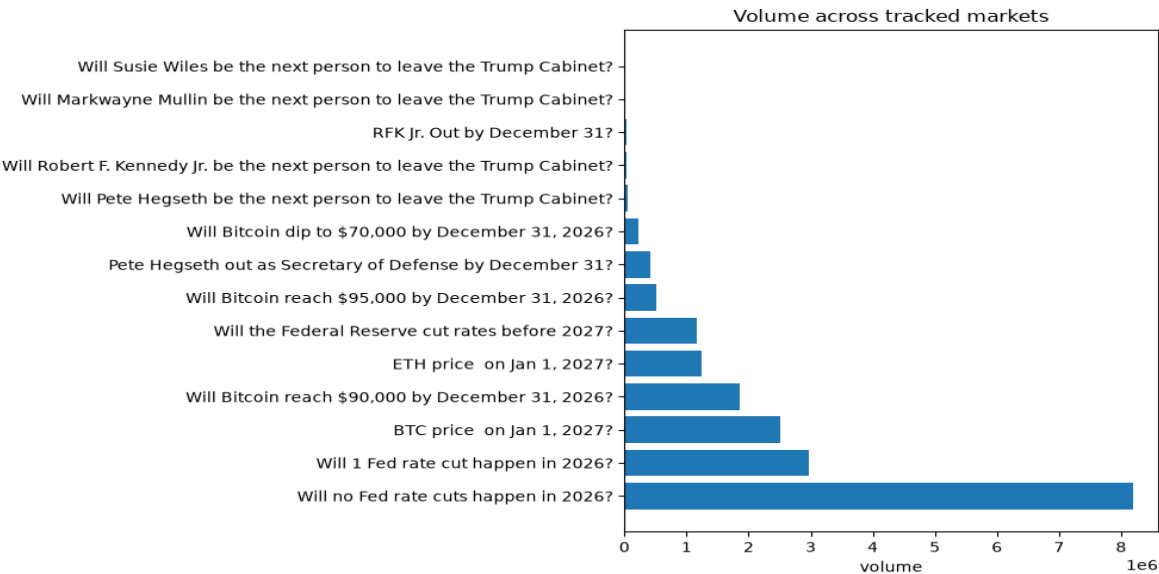
4.1 Repricing speed ("prediction-market VIX")

This operationalizes narrator's framing: mean absolute price change per hour, per market, over the observation window.



Crypto threshold markets and politics markets with active reshuffling, like "Will Markwayne Mullin be the next to leave the Cabinet?", move fastest. The near-consensus "no Fed rate cuts" market, which is also the highest-volume market tracked, moves slowest. That fits the volume-reliability finding below: high conviction and high volume line up with low repricing.

4.2 Volume distribution

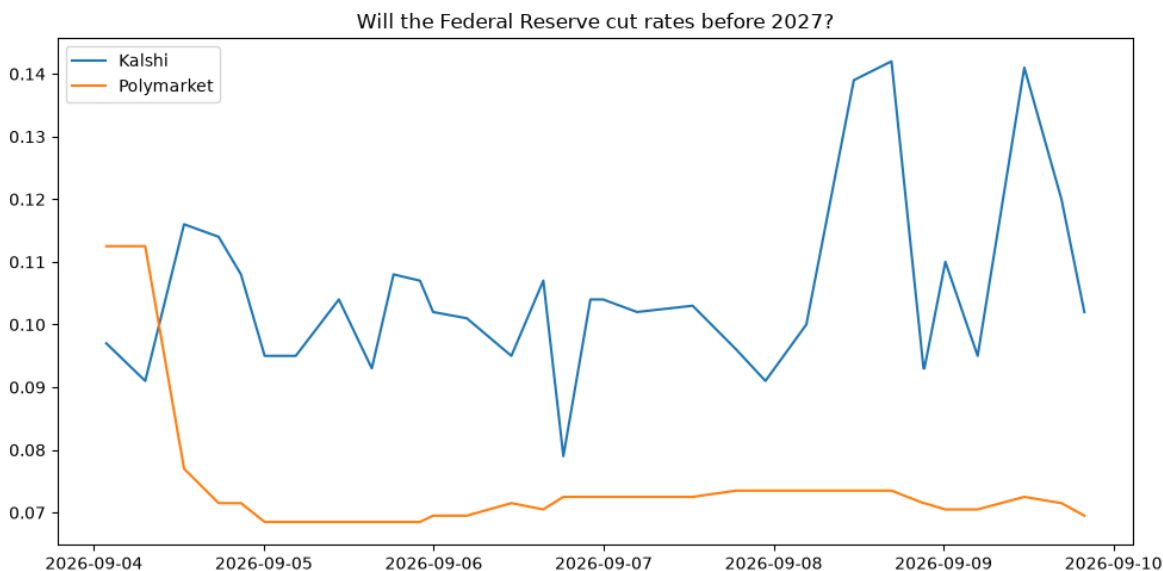


The Polymarket "Will no Fed rate cuts happen in 2026?" market carries about \$8.19M in volume, against near-zero volume on niche Kalshi cabinet markets like Susie Wiles and Markwayne Mullin. That lines up with Evercore ISI's analysis of five years of completed Kalshi and Polymarket markets, which found high-volume markets price more reliably than thin ones, here in a self-collected instance rather than their original sample.

4.3 Cross-platform divergence

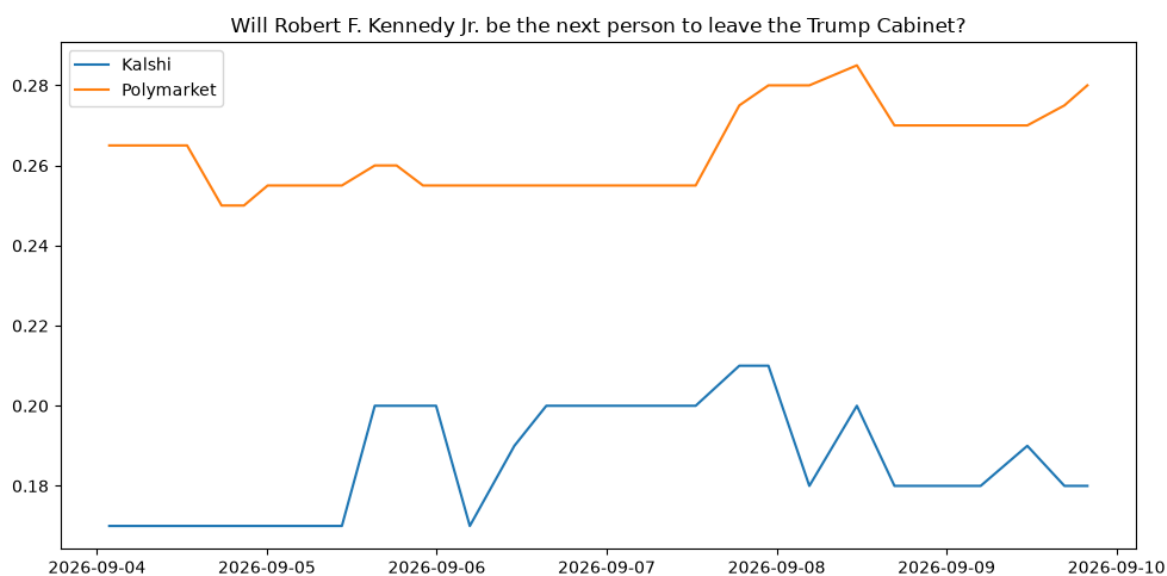
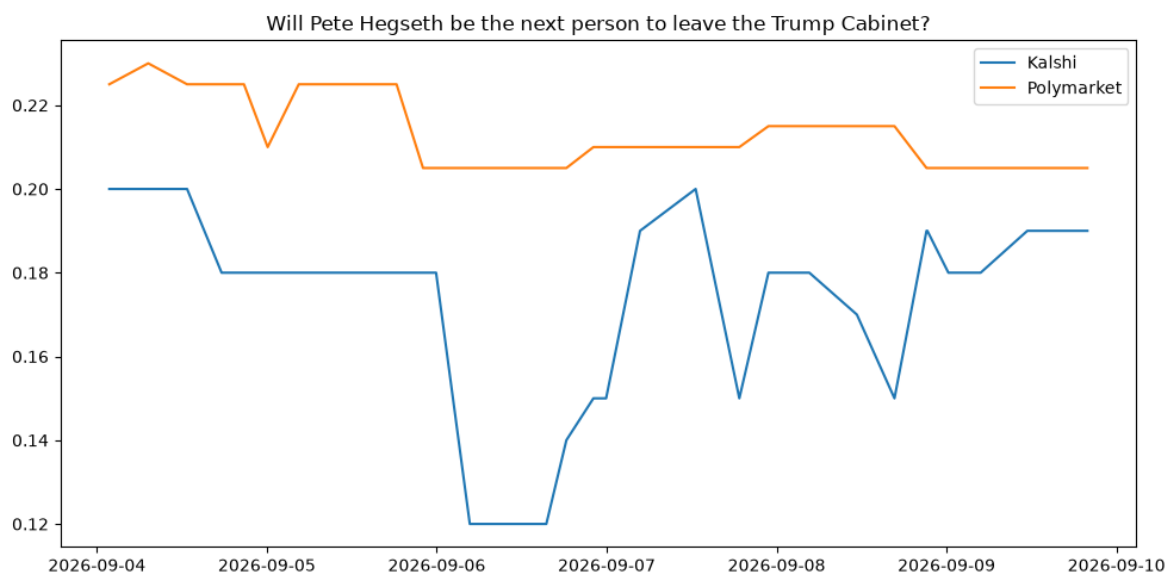
Of five intended cross-platform pairs, only three are populated in `cross_platform_links`: one macro pair and two politics pairs. I didn't add crypto pairs. Checking the actual contract terms, not just the dollar strikes, showed that the tracked Kalshi crypto markets ask what price BTC or ETH will be on a specific future date, while the tracked Polymarket markets ask whether that price is ever touched before a deadline. Those are different questions even at matching strikes, so linking them would have produced a divergence number that looks real but doesn't measure anything.

Macro pair: Fed rate cut



This compares Kalshi's "will the Fed cut rates before 2027" against Polymarket's "will no Fed rate cuts happen in 2026," with Polymarket's series inverted for comparison. That inversion mattered: the two questions are logically opposite (on Kalshi, Yes means a cut happens; on Polymarket, Yes means it doesn't), so a naive raw comparison showed a false ~80-percentage-point "divergence" that was really just two inverted questions lined up against each other. Once I corrected for polarity, the two series started close together on September 4, Kalshi at 0.097 and Polymarket at 0.113, then separated over the following days. Kalshi ended at 0.102, roughly where it started, after a mid-window spike to 0.142. Polymarket declined steadily to 0.070. The gap widened mostly because Polymarket drifted down, not because Kalshi moved up. Across 32 aligned snapshots the Kalshi:Polymarket ratio averaged 1.44 (median 1.43, standard deviation 0.24, range 0.81 to 1.95). I can't tell from one week of data whether that's a real information or liquidity asymmetry between the platforms, or just noise from a short observation window.

Politics pairs



The two politics pairs don't show one uniform pattern, and treating them as if they did would overstate the finding. The RFK Jr. pair (Kalshi:Polymarket ratio) is tight and consistent: mean 0.71, median 0.68, standard deviation 0.06 across 32 points, ranging only from 0.64 to 0.78. Kalshi is consistently pricing this market about 30% below Polymarket. The Pete Hegseth pair is both higher and noisier: mean 0.81, median 0.84, standard deviation 0.11, ranging from 0.59 to 0.95, with no comparably tight relationship. These are two distinct patterns, not one "~65-70%, consistent" finding across politics.

5. Limitations

- About 19 markets tracked, a deliberately small and fixed watchlist, not the full catalog of either platform.
- Only 3 of 5 planned cross-platform links are populated. The 2 crypto pairs were never added: the tracked Kalshi and Polymarket crypto markets turned out to be different contract types (price-on-a-date versus touch-by-deadline), not just different strikes.
- Eight days of observation (2026-09-02 to 2026-09-09), and only six for the cross-platform pairs. The Fed-pair divergence is a real, measured result, but it's too short a window to call it a durable pattern rather than a one-week move. The mid-window spike to 0.142 on the Kalshi side, which had receded by the end of the window, is a good illustration of how much a single week can mislead.
- The politics pairs are approximate matches. Kalshi's cabinet-departure tickers resolve by May 22, 2026, while the matched Polymarket markets resolve by December 31, 2026: the same underlying question, but different windows.
- Polymarket's open_interest isn't available at market granularity (see Section 2), so it's NULL throughout.

6. Next steps

- Track a genuinely comparable crypto pair. This needs a Kalshi market phrased as touch-by-deadline, since the ones currently tracked ask a different question than the Polymarket side.
- Trade-level ingestion and wallet-level "smart money" tracking through Polymarket's subgraph. This was scoped out of this week's MVP and flagged as a stretch goal from the start.
- Widen the watchlist once the pipeline has run unattended for longer, to strengthen the volume-reliability and repricing-speed findings with a bigger sample.